

Computational Methodology

PAWSEY PARTNER SCHEME

Please follow the guidelines to complete each section. Maximum 1000 words in total.

Part 1: Project Title

Multiphase Astrophysics to Unveil the Virgo Environment - MAUVE

Part 2: Suitability

Please indicate why you need access to the supercomputer. Organise your description following instructions below.

Technical Details

This project is centred on the use of the *new Galaxy Integral-field Spectroscopic Tool* (nGIST), a purpose-built analysis framework designed to extract science-ready, value-added data products from MUSE datacubes. These include stellar and ionised gas kinematics, individual emission-line intensity maps, star formation histories, stellar metallicities, and extinction maps. nGIST builds on the publicly available GIST pipeline (Bittner et al. 2019) but has been substantially redesigned to address the computational and scientific demands of modern IFS surveys. More details can be found in Fraser-McKelvie et al. 2025.

A typical MAUVE-MUSE datacube is ~30 GB in size, containing ~750,000 spaxels across 3,761 spectral channels. Each full analysis requires roughly 384 CPU-hours and an average memory allocation of 200 Gb. These numbers have been tested on both the Canadian Advanced Network for Astronomical Research (CANFAR) and the OzStar HPC facility at Swinburne University. For this project, we will analyse 40 galaxies with 120 different realisations each (spanning template libraries, wavelength ranges, and binning schemes), resulting in ~4,800 independent jobs.

Our workflow is embarrassingly parallel: each galaxy–realisation pair is independent, enabling efficient scaling to thousands of simultaneous jobs. Pre-processing involves cube preparation and foreground/background source masking; post-processing aggregates the results into final kinematic, metallicity, and SFH maps. The approach is highly modular, meaning that the routing can restart from where it stopped in case of wall-time issues or other unexpected challenges.

nGIST is written primarily in Python but makes extensive use of optimised numerical libraries (NumPy, SciPy, h5py) and compiled routines through C/C++ bindings. Parallelisation has been re-engineered to use Joblib, replacing the default multiprocessing package. This provides more efficient shared-memory usage, reduces serialization overhead, and improves load balancing. Additional recent optimisations include: Voronoi binning acceleration using SciPy's cKDTree for nearest-neighbour searches, cutting both runtime and RAM footprint; HDF5-based I/O for binned and unbinned spectra, replacing FITS tables and enabling efficient out-of-core operations on large datasets; Optimised scheduling and parallelisation for large multi-core runs, improving throughput across thousands of jobs.

These improvements ensure efficient use of HPC resources by minimising memory bottlenecks, reducing wall-clock times, and enabling near-linear scaling. We also use profiling tools (e.g., Allinea MAP) to identify hotspots and continue to refine the pipeline. This balance of optimised compiled routines with Python's flexibility for rapid prototyping demonstrates that nGIST makes efficient and responsible use of finite HPC resources, while enabling a project of unprecedented scale in IFS data analysis.

Resource Justification

This project cannot be carried out on institutional or commercial cloud clusters due to the combination of scale, workflow structure, and storage requirements. While each job can run on modest node sizes (16 CPUs), the project's scale means that thousands of concurrent jobs must be run in parallel to complete the work in a reasonable timeframe. Institutional clusters do not have the required throughput or storage capacity, and commercial cloud solutions would be prohibitively expensive at this scale.

The workflow is embarrassingly parallel, with independent jobs that can be efficiently distributed across thousands of cores. This allows us to take full advantage of Pawsey's scheduling, job array management, and high-throughput compute capabilities. On a smaller cluster, these jobs would be forced into serial or low-parallel throughput, increasing wall-clock time by years.

The average MAUVE cube does require 384 CPU-hours and 200 GB of RAM for a single nGIST run, corresponding to 384 SU. To reach our goals, we require 120 realisations for each of the 40 MAUVE galaxies for a total number of 1843k service units. Adding a contingency buffer for testing and trial-and-error opportunities we round up this requirement to **2000 kSUs**.

The project is also data-intensive. Each realisation requires around 200 Gb of scratch disc space and 20 Gb for final storage. As such, to efficiently complete all realisations for one galaxy at a time we require 24 TB of disc space on scratch and 3 TB of storage on Acacia. This is essential for handling intermediate products and repeated realisations of MUSE datacubes.

Finally, we reiterate that we are actively optimising nGIST for efficient HPC use: implementing memory-efficient binning algorithms, transitioning to HDF5 I/O, and refining parallelisation. These steps ensure that the resource is not just usable, but necessary, and that we will exploit Pawsey's capabilities to deliver science outcomes that could not be achieved elsewhere.

Does the resource support your workflow?

Our workflow is fully compatible with Setonix's 24-hour maximum wall time. A single nGIST realization of a MUSE datacube typically completes in 3 hours on 126 cores. Even under conservative assumptions, each job finishes comfortably within the 24-hour limit.

More importantly, nGIST is designed to be highly modular. The pipeline is structured as a series of independent modules — e.g. data cube pre-processing, Voronoi binning, stellar kinematics fitting, emission line modeling, and star formation history reconstruction. Each module can be run independently and writes its outputs (in HDF5 format) to disk before moving to the next stage. This means that the workflow has a natural checkpoint/restart mechanism: if a job is interrupted, it can restart from the last completed module rather than reprocessing the entire datacube.

As a result, wall-time limits are not a constraint. No single job requires more than 24 hours, and the modular structure ensures that partial progress is always preserved. For the full survey, we exploit the embarrassingly parallel nature of the problem, with 4,800 realizations to be run independently, making efficient use of job arrays on Setonix.

Part 3: Scalability

This overlaps with the suitability criteria, but particularly addresses the question of why you need to use a national peak supercomputer, and whether you can use it efficiently. Performance and scalability data is historically lacking from many merit applications, and thus sets some applications above the rest. Further to this, performance data

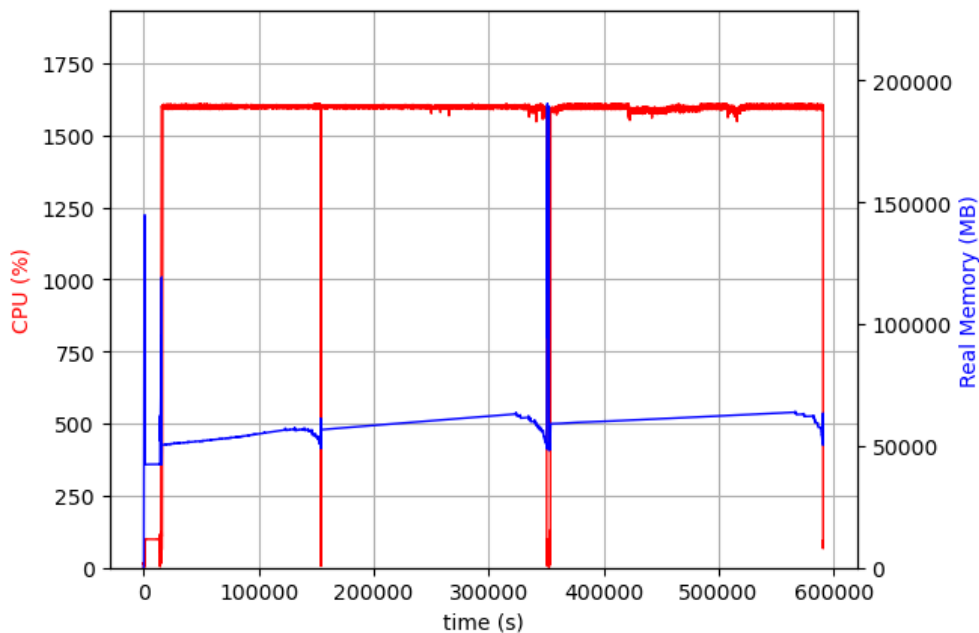
based on a typical job you will run brings confidence to the requested allocation, as opposed to the requested amount appearing to be guessed or just based on last year's usage. Follow instructions and example below.

The nGIST pipeline is designed to be embarrassingly parallel. Each spaxel spectrum within a datacube is treated independently, so the workflow can distribute the fitting of thousands of spectra across available cores without inter-process communication. This results in near-linear scalability: running on N , $2 \times N$, ... $n \times N$ cores yields essentially identical efficiency. This scaling behaviour is particularly well suited to Setonix, where job arrays can be used to submit thousands of independent tasks concurrently.

The only component that does not scale perfectly is the input/output stage. Reading the full datacube into memory is inherently serial and therefore introduces a minor bottleneck. However, this stage represents only a very small fraction of the total runtime. As shown in the figure below, the vast majority of computational time is spent on the fitting modules, which are fully parallelised. Once the data are read, all subsequent steps scale efficiently across cores.

In practice, this means that nGIST achieves high utilisation of HPC resources and makes efficient use of finite allocation. The modular structure of the workflow (pre-processing, binning, stellar fitting, emission-line fitting, SFH reconstruction) allows individual modules to be scheduled and parallelised independently, further improving throughput.

Example of CPU and memory usage for one of the largest MAUVE cubes



Part 4: Usage *(this section is not required for new projects)*

Explain unused past allocations, or large increases in usage. Follow instructions below.

N/A

Part 5: Appropriateness of Request

Justify requested resources. Follow instructions below.

The request of this proposal is specified in part 2. Estimates are based on our experience running nGIST on the OzStar HPC facility at Swinburne and on the CANFAR platform. Depending on the size of cube, we expect jobs to vary in length, but wall-time will be shorter than the maximum wall time of 24 hours. While we plan to run 120 realisations for each cube, run time will be practically identical for all stellar population synthesis models, while will scale as function of the size of the wavelength range and binning scheme used for fitting. As such, our estimates are based on the average MAUVE cube size, and intermediate wavelength range and binning scheme, while the minimum and maximum job sizes provided in the cover of this proposal provide an idea of the range of job sizes we expect.

